tag is encoded and appended to the database ID to return to the client. Only the original token ID is stored in the database, and an attacker without access to the secret key cannot calculate a valid authentication tag.

In this section, you'll authenticate the database tokens with the widely used *HMAC-SHA256* algorithm. HMAC-SHA256 takes a 256-bit secret key and an input message and produces a 256-bit authentication tag. There are many wrong ways to construct a secure MAC from a hash function, so rather than trying to build your own solution you should always use HMAC, which has been extensively studied by experts. For more information on how to build a secure MAC algorithm, I recommend *Serious Cryptography* by Jean-Philippe Aumasson (2017, No starch press).

Rather than storing the authentication tag in the database alongside the token ID, you'll instead leave that as it is. Before you return the token ID to the client, you'll compute the HMAC tag and append it to the encoded token, as shown in figure 5.9. When the client sends a request back to the API including the token, you can validate the authentication tag. If it is valid, then the tag is stripped off and the original token ID passed to the database token store. If the tag is invalid or missing, then the request can be immediately rejected without any database lookups, preventing any timing attacks. Because an attacker with access to the database cannot create a valid authentication tag, they can't use any stolen tokens to access the API and they can't create their own tokens by inserting records into the database.